초보자 실습 강의

Harness 입문

한 문장으로 Claude Code 에이전트 팀을 만든다

revfactory/harness v2.1 — 설치에서 내 업무 하네스까지

에이전트 팀의 개념을 익힌 뒤, 프롬프트만으로 하네스를 설치하고 만들고 실행하고 고쳐 본다.
모든 실습은 프롬프트를 붙여넣고 결과를 확인하는 방식으로 진행한다.

하네스는 일을 맡길 팀을 미리 짜 두는 일이다

강의가 끝났을 때 손에 남아야 할 세 가지를 먼저 정리한다

1

한 명보다 팀이 낫다

에이전트 하나가 쓰고 검토까지 하면 놓치는 것이 생긴다. 역할을 나눈 팀이 품질을 끌어올린다.

2

팀은 한 문장으로 만든다

“하네스 구성해줘” 한 문장이면 Harness가 에이전트 파일과 스킬 파일을 자동으로 만든다.

3

만든 뒤에는 회고로 키운다

써 보고 아쉬운 점을 말하면 하네스가 스스로 고쳐, 다음 실행이 더 나아진다.

수강생이 할 일은 코드를 쓰는 것이 아니라, 원하는 바를 말하고 결과를 검토하는 것이다

강의는 개념에서 시작해 내 업무 하네스로 끝난다

네 단계를 차례로 밟으며, 실습마다 프롬프트를 직접 붙여넣는다

1 기본 지식

에이전트와 스킬

팀이 왜 필요한지 이해하고 꼭 필요한 용어를 익힌다

슬라이드 5~10

2 구조 이해

Harness가 만드는 것

한 문장이 어떤 파일과 팀 구성으로 바뀌는지 본다

슬라이드 12~19

3 따라 하기

실습 0~6

설치하고, 첫 팀을 만들고, 실행하고, 회고까지 해 본다

슬라이드 21~30

4 내 하네스

실습 7~8

내 업무와 금융 리스크 평가 팀을 직접 설계한다

슬라이드 32~38

필요한 것: Claude Code가 설치된 컴퓨터, 인터넷 연결, 붙여넣기 할 손

01

PART 1 · 기본 지식

에이전트와 스킬, 그리고 하네스

새 개념을 쓰기 전에 정의와 그것이 필요한 이유부터 확인한다

에이전트는 답만 하지 않고 직접 일을 끝낸다

agent = 도구를 써서 실제 작업을 수행하는 AI. 챗봇과 무엇이 다른지 비교한다

Claude Code = 내 컴퓨터의 프로젝트 폴더 안에서 일하는 에이전트

챗봇

- 질문을 받으면 답을 글로 돌려준다
- 파일 저장과 실행은 사람이 직접 한다
- 결과가 맞는지 확인하는 일도 사람 몫이다

에이전트 (Claude Code)

- 파일을 읽고 쓰고 명령을 실행한다
- 결과를 보고 오류를 스스로 고친다
- 다른 에이전트에게 일을 나눠 맡긴다

하네스를 이해하려면 다섯 용어를 알아야 한다

모두 프로젝트 폴더 안의 텍스트 파일이다 — 직접 쓰지 않고 Claude가 만든다

용어	정체	쉽게 말하면
서브에이전트 (subagent)	.claude/agents/이름.md	한 가지 일만 맡는 전문 직원
스킬 (skill)	.claude/skills/이름/SKILL.md	일하는 방법을 적은 매뉴얼
오케스트레이터	스킬의 한 종류	누가, 언제, 어떤 순서로 할지 정한 진행표
플러그인 (plugin)	스킬을 묶은 설치 꾸러미	앱 스토어에서 받는 앱
CLAUDE.md	프로젝트 맨 위의 안내 메모	세션을 열 때마다 먼저 읽는 쪽지

Harness는 이 다섯 가지를 한꺼번에 설계하고 만들어 주는 플러그인이다

하네스는 에이전트에게 씌우는 마구다

harness = 말의 힘을 한 방향으로 모으는 마구. AI에서는 미리 짜 둔 작업 구성을 뜻한다

같은 모델이라도, 누가 무엇을 어떤 순서로 할지 미리 정해 두면 결과가 달라진다

하네스가 없을 때

- 매번 긴 지시를 처음부터 다시 쓴다
- 검토 없이 그럴듯한 결과를 받는다
- 잘된 방식이 다음 대화에 남지 않는다

하네스가 있을 때

- 역할과 절차가 파일로 남아 재사용된다
- 검증 담당이 틀린 결과를 걸러 낸다
- 회고할 때마다 방식이 조금씩 나아진다

혼자 쓰고 혼자 검토하면 놓치기 쉽다

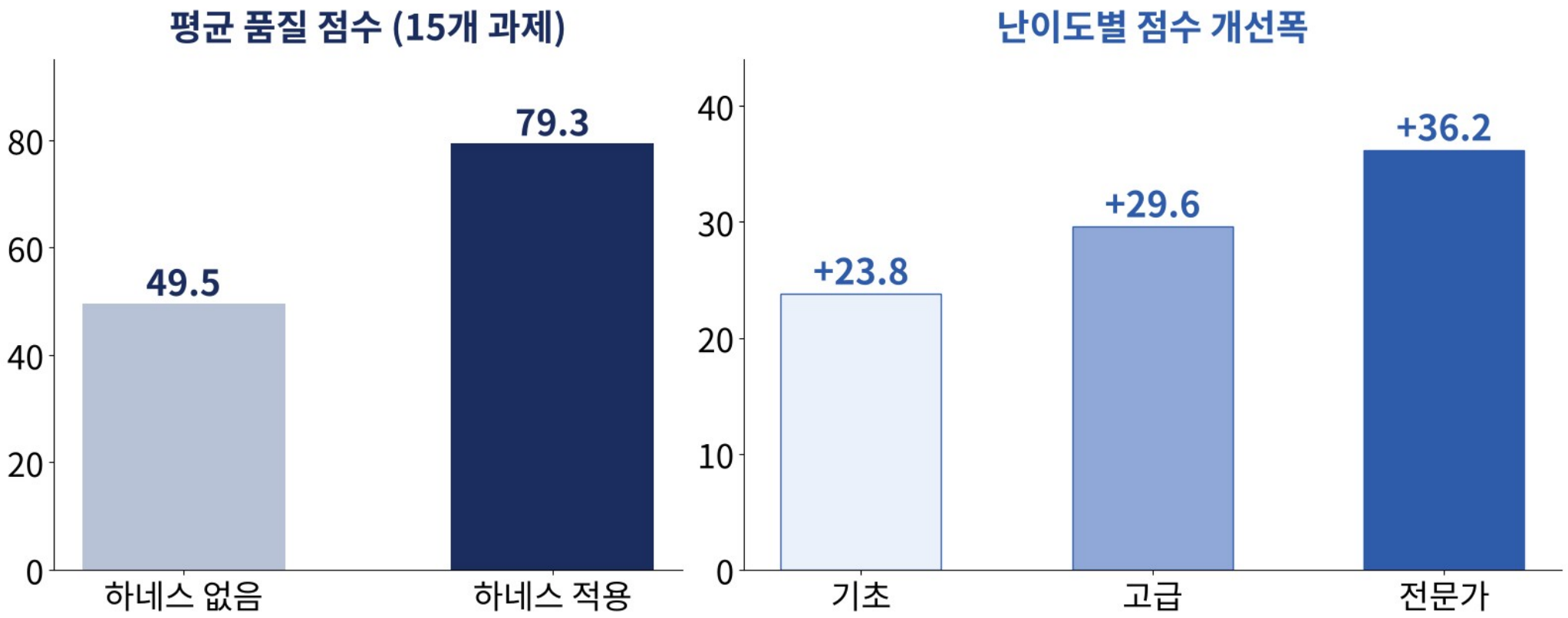
에이전트 하나에 모든 일을 맡길 때 생기는 세 가지 한계와 팀의 해법

한계	왜 생기나	팀의 해법
자기 검토의 맹점	자기가 쓴 글의 오류는 잘 보이지 않는다	다른 에이전트가 반박하며 검증한다
맥락 과부하	한 대화에 모든 일을 넣으면 흐려진다	역할마다 맥락을 따로 나눈다
느린 순차 처리	일을 하나씩 차례로만 처리한다	서로 독립된 일은 동시에 한다

Harness는 이 해법을 업무에 맞는 팀 구조로 자동 설계한다

구조를 미리 짜 두면 결과 품질이 올라간다

저자 자체 A/B 실험(n=15, v1 기준) — 15개 과제 모두 개선됐고 어려울수록 효과가 컸다



출력 편차도 32% 줄었다. 저자가 측정한 수치이므로, 조직 도입 전에는 2~4주 자체 파일럿으로 확인한다

Harness는 Claude Code용 팀 설계 플러그인이다

revfactory/harness — 업무 설명 한 문장을 에이전트 팀과 스킬로 바꾸는 메타 스킬

v2.1.0	9.2k	6 + 3	Apache 2.0
최신 버전 (2026년 9월 26일)	GitHub 스타 수	팀 구성 방식과 실행 모드	무료로 쓰는 오픈소스

메타 스킬 (meta-skill)이란

- 직접 일을 하는 스킬이 아니라, 일할 팀과 그 팀이 쓸 스킬을 만들어 주는 스킬이다
- v2부터는 실험 기능을 켜는 환경 변수 없이 설치만 하면 바로 동작한다

02

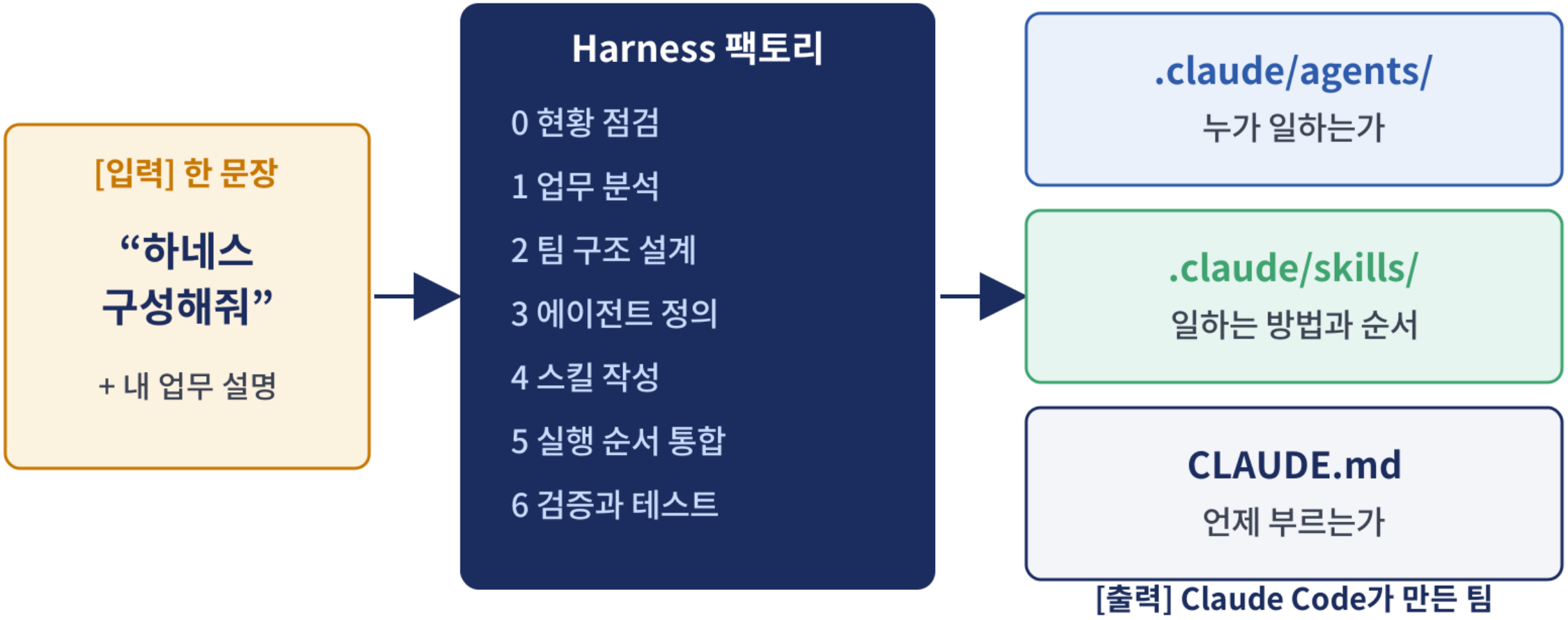
PART 2 · 구조 이해

Harness가 만드는 것

한 문장이 어떤 단계를 거쳐 어떤 파일과 팀 구조로 바뀌는지 살펴본다

한 문장이 일곱 단계를 거쳐 팀이 된다

0~6단계는 Harness가 알아서 진행하고, 사람은 중간에 설계안을 확인만 한다



Harness는 여덟 단계로 팀을 설계하고 관리한다

마지막 7단계(운영과 개선)는 별도의 회고 스킬 harness:evolve가 맡는다

단계	Harness가 하는 일
0 현황 점검	기존 에이전트와 스킬을 읽고, 새로 만들지 넓힐지 정한다
1 업무 분석	어떤 작업이 있고, 일의 흐름이 어떤 모양인지 파악한다
2 팀 구조 설계	세 가지 실행 모드와 여섯 가지 팀 구성 중 맞는 것을 고른다
3 에이전트 정의	.claude/agents/ 아래에 전문가 파일을 만든다
4 스킬 작성	.claude/skills/ 아래에 일하는 방법을 적는다
5 실행 순서 통합	오케스트레이터를 만들고 CLAUDE.md에 호출 조건을 적는다
6 검증과 테스트	파일을 점검하고 모의 실행으로 동작을 확인한다
7 운영과 개선	써 본 결과를 회고해 고친다 (harness:evolve)

결과물은 세 종류의 파일로 남는다

에이전트 파일은 누가, 스킬은 어떻게, CLAUDE.md는 언제 부를지를 적는다

[출력] Harness가 만드는 폴더 구조

```
내-프로젝트/
  .claude/
    agents/          ← 누가
      analyst.md
      builder.md
      qa.md
    skills/          ← 어떻게
      analyze/SKILL.md
      build/SKILL.md
      (오케스트레이터)
  _workspace/        ← 중간 결과
  CLAUDE.md          ← 언제 부르나
```

에이전트 파일에는

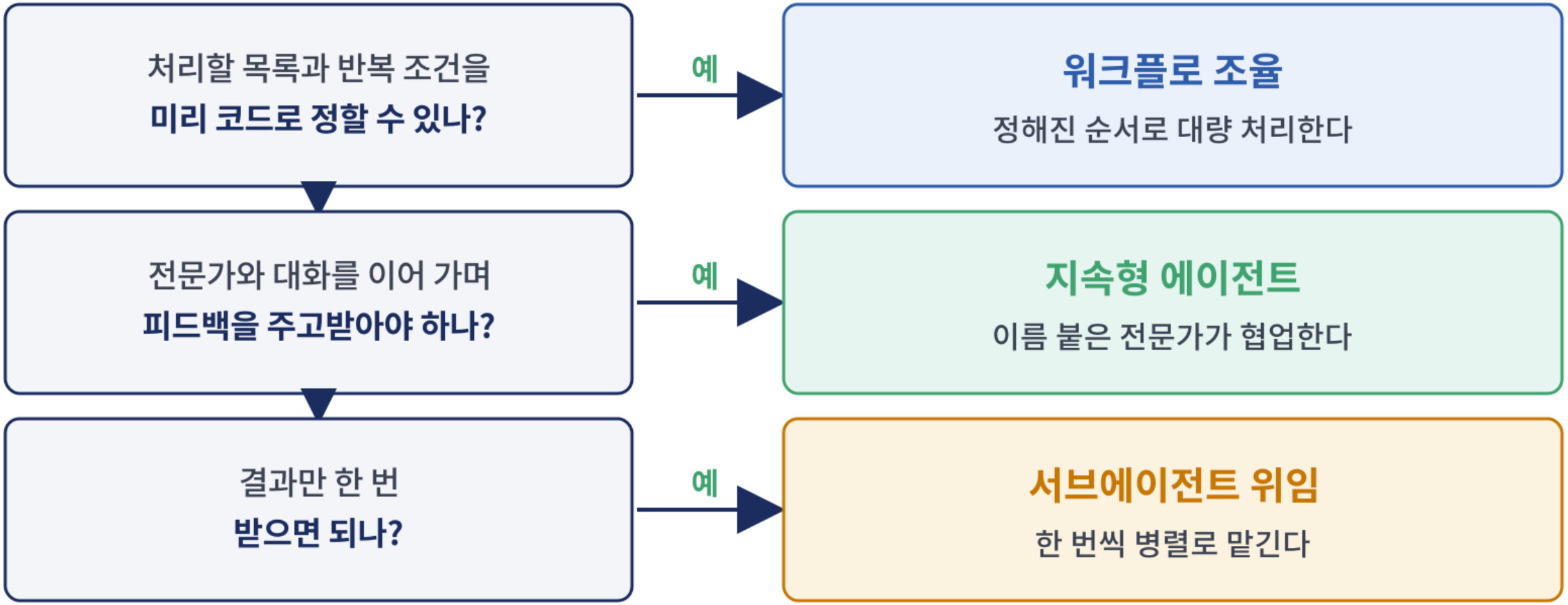
- 맡은 역할과 일하는 원칙
- 쓸 모델과 고른 이유

스킬과 CLAUDE.md에는

- 스킬: 일하는 절차
- CLAUDE.md: 호출 조건만

일의 흐름이 실행 모드를 정한다

위에서부터 차례로 묻는다 — 팀 크기가 아니라 흐름의 모양이 판단 기준이다



팀을 짜는 방식은 여섯 가지다

실제 하네스는 여러 방식을 섞어 쓴다 — 예를 들어 분산·통합으로 조사한 뒤 생성·검증으로 다듬는다

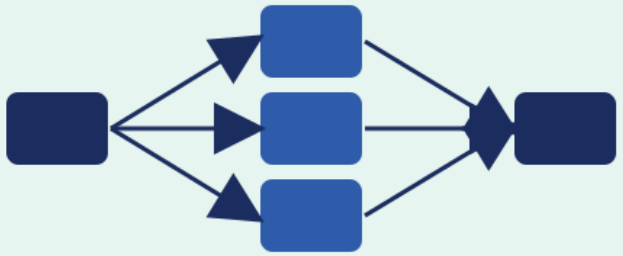
파이프라인

앞 결과를 받아 순서대로



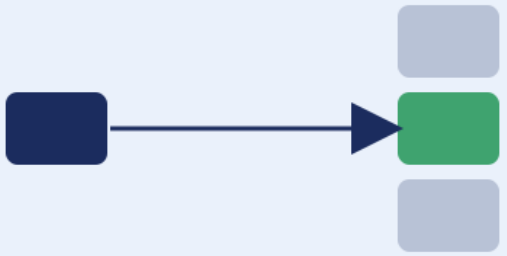
분산·통합

동시에 나눠 하고 합친다



전문가 풀

맞는 전문가만 부른다



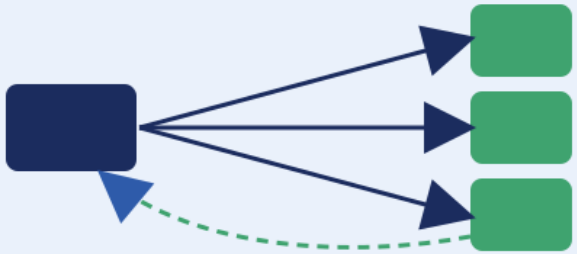
생성·검증

만들고 다른 이가 검토



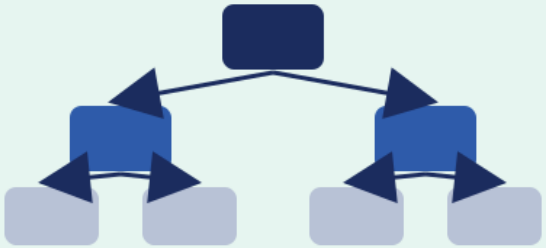
감독자

진행 보며 다시 배분



계층형 위임

팀장이 다시 맡긴다



업무 모양을 보면 어울리는 구성이 보인다

Harness README의 예시 업무가 어떤 팀 구성으로 이어지는지 정리했다

업무 예	어울리는 구성	이유
웹사이트 제작	파이프라인	설계 다음에 구현, 구현 다음에 테스트를 한다
종합 코드 리뷰	분산·통합	보안과 성능, 스타일을 동시에 보고 합친다
고객 문의 처리	전문가 풀	문의 종류에 맞는 담당만 부른다
웹툰 제작	생성·검증	그린 뒤 다른 에이전트가 일관성을 검토한다
유튜브 기획	감독자	진행 상황을 보며 일을 다시 나눈다
데이터 파이프라인	계층형 위임	큰 설계를 팀장 에이전트가 나눠 맡긴다

구성은 Harness가 고른다 — 사람은 왜 그 구성인지 물어보고 납득하면 된다

그럴듯하지만 틀린 결과를 걸러 낸다

v2에서 추가된 품질 검증 방식 — 정확성이 중요한 하네스에 덧붙인다

검증 방식	어떻게 거르나
적대적 검증	찾은 항목마다 반박 담당 여러 명이 확인하고, 과반이 인정한 것만 남긴다
심사위원단	시안을 여러 개 만들어 심사하고, 가장 좋은 안을 바탕으로 최종본을 쓴다
새 항목이 없을 때까지 반복	새로 찾은 것이 여러 번 연속 0건이 될 때까지 탐색한다
여러 기준으로 탐색	서로 다른 기준을 맡은 에이전트들이 동시에 찾는다
누락 검토	마지막에 빠진 것만 찾는 담당을 한 명 둔다

에이전트마다 일에 맞는 모델을 고른다

v2.1 정책 — 중요해 보인다는 이유만으로 모두 최고 모델에 두지 않는다

모델	언제 쓰나	대표 업무
fable	스스로 계획해 오래 자율 실행하는 최고 난도 일	에이전트 조율, 방대한 자료 통합
opus	범위는 분명하지만 깊은 분석이 필요한 일	설계, 코드 생성, 교차 검증
sonnet	절차가 분명하고 빨리 끝내야 하는 일상 업무	형식 변환, 단순 수집, 요약

판단이 애매하면 sonnet이 기본값이다 — 비용과 속도를 함께 아낀다

03

PART 3 · 따라 하기

설치에서 회고까지 실습

모든 실습은 프롬프트를 붙여넣고 결과를 확인하는 방식으로 진행한다

실습 슬라이드는 네 칸으로 읽는다

모든 실습이 같은 배치다 — 칸의 색을 보면 무엇을 해야 할지 알 수 있다

① 어디

파란 띠

프롬프트를 넣을 창.
대부분 Claude Code
대화창이다

② 입력

앰버 박스와 파란 헤더 박스

그대로 복사해 붙여넣을
프롬프트나 명령어

③ 출력

네이비 헤더 박스

Claude가 돌려주는
응답의 예시. 실제 문구는
다를 수 있다

④ 피드백

초록 패널

결과가 다르거나 아쉬울 때
다시 보낼 말

직접 하는 일은 붙여넣기와 확인뿐이다 — 파일 만들기와 오류 해결도 Claude Code에게 시킨다

실습 0 — 빈 프로젝트 폴더를 준비한다

git으로 초기화해 두면 나중에 회고 스킬이 바뀐 내역을 추적할 수 있다

① 어디

터미널에서 시작한다 (Mac은 터미널 앱, Windows는 PowerShell)

[입력] 명령어 · 터미널

```
mkdir harness-lab && cd harness-lab && claude
```

[입력] 프롬프트 · Claude Code 대화창

```
이 폴더를 하네스 실습용 프로젝트로 준비해줘.
git 저장소로 초기화하고 README.md에
'하네스 실습 프로젝트'라고 한 줄 적어줘.
끝나면 폴더 구조를 보여줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
✓ git 저장소를 초기화했습니다
✓ README.md를 만들었습니다

harness-lab/
├──── .git/
└──── README.md
```

④ 피드백

- “claude: command not found”가 나오면 Claude Code가 아직 설치되지 않은 것이다
- git 설정을 물어 오면 “네가 알아서 기본값으로 설정해줘”라고 답한다

실습 1 — Harness 플러그인을 설치한다

Claude Code 대화창에 슬래시(/) 명령 두 줄을 차례로 입력하면 끝난다

① 어디 — **Claude Code 대화창 — 슬래시로 시작하는 명령은 한 줄씩 넣고 Enter를 누른다**

[입력] 명령어 ① · 마켓플레이스 등록

```
/plugin marketplace add revfactory/harness
```

[입력] 명령어 ② · 플러그인 설치

```
/plugin install harness@harness-marketplace
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
✓ 마켓플레이스 harness-marketplace 추가
✓ harness@harness-marketplace 설치

환경 변수나 실험 기능 설정은
필요 없다 (v2부터)
```

④ 피드백

- 설치가 안 보이면 /plugin list 로 확인한다. 비활성이면 /plugin enable harness@harness-marketplace
- 목록에 아예 없으면 ① 명령부터 다시 입력한다

실습 1 확인 — 설치된 스킬에게 자기소개를 시킨다

설치가 됐는지 확인하는 일도 Claude Code에게 맡긴다

① 어디 Claude Code 대화창

[입력] 프롬프트 · Claude Code 대화창

```
harness 플러그인이 제대로 설치됐는지 확인해줘.
설치된 스킬 이름을 알려 주고,
각 스킬이 언제 쓰이는지
초보자에게 세 줄로 설명해줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

설치된 스킬 2개

- harness:harness
  하네스를 새로 만들거나 점검할 때
- harness:evolve
  써 본 하네스를 회고해 고칠 때

“하네스 구성해줘” 라고 하면 시작됩니다

④ 피드백

- 스킬이 안 보이면 “Claude Code를 다시 시작해야 하는지 확인해줘”라고 묻는다
- 설명이 어려우면 “회사 조직에 비유해서 다시 설명해줘”라고 한다

실습 2 — 첫 하네스를 한 문장으로 만든다

Harness README에 실린 리서치 하네스 예시를 초보자용으로 다듬었다

① 어디 — **Claude Code 대화창 (harness-lab 폴더에서 실행 중)**

[입력] 프롬프트 · Claude Code 대화창

```
리서치 하네스를 구성해줘.
어떤 주제든 공식 자료, 언론, 커뮤니티 반응을
여러 각도에서 조사하고, 주장을 교차 검증한 뒤
종합 보고서를 쓰는 팀이 필요해.
나는 코딩 초보야. 파일을 만들기 전에
설계안을 쉬운 말로 먼저 보여 주고
내 확인을 받아줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
[0단계] 기존 하네스 없음 → 새로 구축
[제안] 분산·통합 + 적대적 검증
  실행 모드: 워크플로 조율
  에이전트: researcher,
    fact-checker, synthesizer
이대로 파일을 만들까요?
```

④ 피드백

- 에이전트가 많아 보이면 “3명으로 줄이고 이유를 알려줘”라고 답한다
- 설계가 괜찮으면 “좋아, 그대로 만들어줘”라고 승인한다

실습 3 — 만들어진 파일을 Claude에게 해설시킨다

파일을 하나하나 열어 읽지 않는다 — 표로 정리해 달라고 한다

① 어디 Claude Code 대화창

[입력] 프롬프트 · Claude Code 대화창

```
방금 만든 하네스 파일을 하나씩 열어서
표로 정리해줘. 열은 파일 경로, 맡은 역할,
사용 모델, 그 모델을 고른 이유로 해줘.
마지막에 CLAUDE.md에 무엇이 적혔는지도
보여줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
agents/researcher.md     조사    sonnet
agents/fact-checker.md   반박    opus
agents/synthesizer.md    종합    opus
skills/research-team/    진행 순서

CLAUDE.md: 호출 조건, 변경 이력
```

④ 피드백

- 역할이 겹쳐 보이면 “두 에이전트를 합치는 게 나을지 판단해줘”라고 묻는다
- 모델이 궁금하면 “왜 이 에이전트는 opus인지 쉽게 설명해줘”라고 한다

실습 4 — 만든 팀에게 첫 과제를 맡긴다

트리거 단어를 외우지 않아도 된다 — CLAUDE.md의 호출 조건이 팀을 불러낸다

① 어디 Claude Code 대화창

[입력] 프롬프트 · Claude Code 대화창

```
리서치 팀으로 조사해줘.
주제: 2026년 국내 기업의 AI 에이전트 도입 현황
최종 보고서는 report.md로 저장하고
중간 결과는 _workspace/에 남겨줘.
기본 규모로 진행하고 끝나면 요약해줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
_workspace/
  01_researcher_official.md
  01_researcher_news.md
  02_fact-checker_verdicts.md
report.md   ← 최종 보고서
검증 통과 주장 12건, 기각 3건
```

④ 피드백

- 여러 에이전트가 함께 일하면 과제 하나에 수만~수십만 토큰을 쓴다
- 한도를 정하려면 프롬프트 끝에 “+200k”처럼 토큰 예산을 붙인다

실습 5 — 부족한 부분만 다시 실행한다

오케스트레이터는 전체를 처음부터 돌리지 않고 해당 단계만 다시 실행할 수 있다

① 어디 — Claude Code 대화창 (실습 4를 마친 같은 세션 또는 새 세션)

[입력] 프롬프트 · Claude Code 대화창

```
보고서에서 커뮤니티 반응 부분이 빈약해.
그 조사만 다시 실행해서 report.md에
반영해줘. 나머지 내용은 그대로 둬.
무엇을 바꿨는지 마지막에 알려줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
_workspace/ 확인 → 부분 수정 요청
커뮤니티 조사만 다시 실행합니다
✓ 01_researcher_community.md 갱신
✓ report.md의 해당 장만 수정
```

④ 피드백

- 전체가 다시 돌면 "이전 결과를 바탕으로 그 부분만 다시"라고 범위를 다시 말한다
- 새 주제를 맡기면 이전 _workspace/는 날짜가 붙은 폴더로 보관된다

회고는 하네스를 다음 세대로 키운다

피드백 하나에 규칙 하나를 붙이지 않고, 원인을 찾아 원리 수준으로 고친다



실습 6 — 하네스를 회고하고 고친다

아쉬웠던 점을 구체적으로 말할수록 고치는 위치가 정확해진다

① 어디 Claude Code 대화창

[입력] 프롬프트 · Claude Code 대화창

```
하네스 회고해줘.
이번 실행에서 커뮤니티 조사가 약했고,
출처 표기 방식이 에이전트마다 달랐어.
이 피드백을 일반화해서 에이전트와 스킬에
반영하고, CLAUDE.md 변경 이력에 기록해줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
[변화량] 처음 구성 대비 변경 2건
[반영] 관점마다 최소 출처 수 기준
[반영] 공통 출처 표기 형식 추가
[기록] CLAUDE.md 변경 이력 1행
[다음 실행] 출처 형식이 통일됩니다
```

④ 피드백

- 변경은 한 번에 하나씩 반영하고 확인한다 — 몰아서 고치면 무엇이 효과였는지 모른다
- 예전 수정과 충돌하면 회고 스킬이 먼저 알려 준다. 둘 다 만족하는 기준을 함께 정한다

04

PART 4 · 내 하네스

내 업무에 맞는 팀 설계

업무 설명을 잘 쓰는 공식을 익히고, 금융 업무와 다른 사례로 직접 설계한다

좋은 하네스 요청은 다섯 가지를 담는다

Harness는 업무 설명을 읽고 팀을 짠다 — 설명이 구체적일수록 팀이 정확해진다

항목	적을 내용	예 (투자 검토)
목표	이 팀이 끝내야 할 일	투자 검토 메모를 쓴다
입력	팀이 받는 자료	기업 IR 자료와 공시
산출물	최종 결과물의 형식과 위치	1페이지 메모, memo.md
역할	필요한 전문가	재무 분석, 리스크, 최종 검토
검증	반드시 확인할 것	숫자 출처와 계산을 다시 확인

규모를 정하고 싶으면 “기본 규모로” 또는 “철저히”를 덧붙인다 — 철저히는 비용도 커진다

실습 7 — 내 업무 하네스를 요청한다

아래 틀의 대괄호 부분만 내 업무로 바꿔서 붙여넣는다

① 어디

Claude Code 대화창 — 새 폴더에서 시작하려면 실습 0의 명령을 이름만 바꿔 다시 쓴다

[입력] 프롬프트 · Claude Code 대화창

```
[업무 이름] 하네스를 구성해줘.
목표: [이 팀이 끝내야 할 일]
입력: [팀이 받는 자료]
산출물: [결과물 형식과 저장 위치]
역할: [필요한 전문가 2~4명]
검증: [반드시 확인할 것]
설계안을 쉬운 말로 먼저 보여 주고,
내 확인을 받은 뒤 파일을 만들어줘.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
[0단계] 현재 폴더 점검
  기존 하네스가 있으면 확장을 제안
[제안] 팀 구성과 실행 모드
[제안] 에이전트별 모델과 이유

승인하시면 파일을 만듭니다.
```

④ 피드백

- 설계가 업무와 다르면 "검토 단계가 빠졌어, 추가해줘"처럼 짚어서 말한다
- 만든 뒤에는 실습 3처럼 표로 해설을 받고, 실습 4처럼 작은 과제부터 맡긴다

실습 8 — 금융 리스크 평가 팀을 만들어 본다

Harness 공식 quickstart 예시 — 리스크 업무는 보통 생성·검증이나 전문가 풀로 짜인다

① 어디

Claude Code 대화창 — 위 박스로 팀을 만든 뒤, 아래 박스로 과제를 맡긴다

[입력] 프롬프트 · Claude Code 대화창 · 팀 만들기

```
핀테크 리스크 평가 팀 하네스 구성해줘.
```

[입력] 프롬프트 · Claude Code 대화창 · 과제 맡기기

```
Ticket FIN-427: 신규 법인 고객(한국 중견
제조사, 매출 $80M)이 $5M 운전자금 한도를
신청했다. 신용 이력의 적신호, 기존 포트폴리오
대비 섹터 집중도, 규제 노출을 따져
go/no-go 권고가 담긴 1페이지 메모를 써라.
```

[출력] 예시 응답 · 실제 문구는 다를 수 있다

```
[구성] 생성·검증
  credit-analyst
  sector-analyst
  regulatory-reviewer
  memo-writer
[산출] memo_FIN-427.md
```

④ 피드백

- 실제 고객 자료를 넣기 전에 회사의 데이터 반출 규정부터 확인한다
- 권고는 초안일 뿐이다 — 최종 판단과 책임은 사람에게 있다

사례 — 웹툰 팀은 생성·검증으로 짠다

Harness 공식 예시 5 — 그리는 에이전트와 검토하는 에이전트 한 쌍이면 충분하다

1 생성

artist · 이름 붙인 지속형

패널을 그려 _workspace/panels/에 저장한다

2 검토

webtoon-reviewer · 단발

패널마다 통과, 수정, 재작업 중 하나로 판정한다

3 재작업

최대 2회

재작업 판정을 받은 패널만 artist에게 다시 맡긴다

설계 포인트

- 다시 고칠 에이전트는 처음부터 이름을 붙여 실행한다 — 이전 맥락을 기억한다
- 패널의 절반 이상이 재작업이면 프롬프트 자체를 고치자고 사람에게 제안한다

사례 — 소설 팀은 지속형과 단발을 섞는다

Harness 공식 예시 2 — 단계마다 일의 성격이 달라 실행 모드도 다르게 고른다

1 설정

지속형 3명 병렬

세계관, 인물, 줄거리 담당이 충돌하면 대화로 조정한다

2 집필

서브에이전트

_workspace/의 설정 파일 세 개를 읽고 원고를 쓴다

3 검토

서브에이전트 병렬

과학 고증과 이야기 연속성을 따로 본다

4 수정

지속형

이름 붙여 실행한 작가에게만 수정을 다시 보낸다

고칠 일이 생길 에이전트에는 처음부터 이름을 붙여 맥락을 이어 간다

완성된 하네스 100개를 참고할 수 있다

revfactory/harness-100 — 모두 이 플러그인으로 만든 실전용 예시 묶음

10	100	4~5명	1,808
업무 분야 (콘텐츠, 개발, 법률 등)	하네스 (한·영 200개 패키지)	하네스마다 전문 에이전트 수	마크다운 파일 수

이렇게 활용한다

- 내 업무와 비슷한 하네스를 찾아, 어떤 팀 구성을 썼는지 먼저 본다
- “이 하네스를 참고해서 내 업무용으로 바꿔줘”라고 Claude Code에 요청한다

막히면 Claude Code에 이렇게 말한다

초보자가 자주 겪는 문제와 그 원인, 그리고 보낼 한마디

증상	원인	이렇게 요청한다
설치했는데 안 불린다	트리거 말이 빠졌다	“하네스 구성해줘”를 문장에 넣는다
토큰을 너무 많이 쓴다	실행 규모가 크다	“기본 규모로, +200k 안에서 해줘”
인터넷 글의 설정과 다르다	v1 문서를 봤다	“하네스 점검해줘” — v2는 실험 설정이 필요 없다
워크플로 사용 동의를 묻는다	Workflow 도구는 동의가 필요하다	내가 요청한 실행이 맞으면 승인한다
결과가 매번 아쉽다	피드백이 반영되지 않았다	“하네스 회고해줘”로 고친다

토론 질문으로 내 업무에 적용해 본다

정답보다 내 업무의 구체적인 장면을 떠올리며 답한다

Q1	내 업무 중 매번 같은 지시를 반복하는 일은 무엇이고, 어떤 역할로 나눌 수 있는가?	**팀 설계**
Q2	검증 담당이 없을 때 그럴듯하지만 틀린 결과가 나오면 어떤 손실이 생기는가?	**품질 검증**
Q3	모든 에이전트를 최고 모델로 두면 무엇이 나빠지는가?	**모델 선택**
Q4	에이전트 팀이 만든 결과의 최종 책임은 누가, 어떻게 져야 하는가?	**사람의 몫**

정리 — 말하고, 확인하고, 회고한다

다음 시간에는 각자 만든 하네스를 실제 과제에 돌려 결과를 함께 검토한다

1

개념

하네스는 누가, 어떻게, 언제 일할지를 미리 파일로 짜 둔 에이전트 팀이다.

2

만들기

“하네스 구성해줘”와 업무 설명 다섯 가지면 Harness가 팀과 스킬을 만든다.

3

키우기

써 보고 “하네스 회고해줘”로 고치면 다음 실행은 더 나은 초안에서 시작한다.

다음 시간 전까지 실습 1의 두 줄로 내 컴퓨터에 Harness를 설치해 둔다

참고 자료

강의 내용은 2026년 10월 기준 저장소(v2.1.0)를 따른다

자료	주소
Harness 저장소 (README, quickstart)	github.com/revfactory/harness
2분 인터랙티브 소개 애니메이션	revfactory.github.io/harness-animation
완성 하네스 100선	github.com/revfactory/harness-100
A/B 실험 (n=15, 저자 측정)	github.com/revfactory/claude-code-harness
Codex용 같은 개념의 도구	github.com/SaehwanPark/meta-harness